SQL LAB EXAM · STUDY NOTES

The Video Course in Text

Every slide from the 9 videos with its recipe, template and explanation. Orange text = blanks you fill in.

Part 1 · Exam Game Plan + CREATE TABLE & Constraints

Part 2 · SELECT Queries, GROUP BY, HAVING & Subqueries

Part 3 · JOINs

Part 4 · Transactions (COMMIT/ROLLBACK/SAVEPOINT) + GRANT/REVOKE

Part 5 · Views & Indexes

Part 6 · Stored Procedures & Functions

Part 7 · Triggers (INSERT / UPDATE / DELETE)

Part 8 · XML Document + XSD Validation

Part 9 · How to Attack ANY Question + Final Revision

Part 1 · Exam Game Plan + CREATE TABLE & Constraints

Hi! Welcome to your SQL lab exam crash course. Relax, you don't need to memorize every question. Almost every question in your papers is built from about ten patterns. In these videos I'll teach you each pattern as a simple recipe, so even if the table names change in the exam, you can still write the answer. Let's start.

What your exam really asks

- 1 · Create tables with constraints (PK, NOT NULL, UNIQUE, CHECK, DEFAULT, FK)
- 2 · Insert / Update / Delete rows
- 3 · SELECT queries: WHERE, ORDER BY, GROUP BY, HAVING, subqueries
- 4 · JOINs: inner, left, full outer, cross, self
- 5 · Transactions: COMMIT, ROLLBACK, SAVEPOINT
- 6 · GRANT / REVOKE permissions
- 7 · Views & Indexes
- 8 · Stored procedures & functions · 9 · Triggers · 10 · XML + XSD

Here's the whole syllabus in one slide. One: creating tables with constraints. Two: insert, update, delete. Three: select queries with where, order by, group by, having, and subqueries. Four: joins. Five: transactions, meaning commit, rollback and savepoint. Six: grant

and revoke. Seven: views and indexes. Eight: stored procedures and functions. Nine: triggers. And ten: X M L with X S D. Each question in your paper is just two or three of these glued together. So if you know these ten recipes, you're ready.

Golden rules for the exam

- End EVERY statement with a semicolon ;
- Text and dates go in single quotes: 'Chennai' '2024-05-10'
- Numbers have NO quotes: 8.5 75000
- Create PARENT tables first, CHILD tables (with FK) after
- Insert parent rows before child rows
- Always show output: SELECT * FROM table; after a change

Before we write any code, six golden rules. First, every statement ends with a semicolon. Second, text and dates go inside single quotes, like quote Chennai quote. Third, numbers never get quotes. Fourth, create the parent table first and the child table, the one with the foreign key, after it. Fifth, insert parent rows before child rows, otherwise the foreign key will reject them. And sixth, after any change, run select star from the table, to show the examiner the result. These rules alone will save you from most errors.

Step 0 · Start every answer like this

1. CREATE DATABASE once, then USE it
2. DESC shows columns, types and keys

```sql
CREATE DATABASE college;
USE college;

-- check what tables exist
SHOW TABLES;
-- see a table's structure
DESC Student;
```

Every answer starts the same way. Create a database, give it any name, like college. Then type USE college, so all your tables go inside it. Show tables lists the tables you have, and DESC, followed by a table name, shows its columns and keys. DESC is a great way to prove to the examiner that your constraints were created.

Data types you need

- INT → IDs, counts, seats, quantity, credits
- VARCHAR(50) → names, city, dept, genre (text)
- DECIMAL(10,2) → money: salary, fare, balance, price
- DECIMAL(4,2) → CGPA like 8.75 · DECIMAL(2,1) → rating like 4.5
- DATE → HireDate, VisitDate written as 'YYYY-MM-DD'
- If an ID looks like 'C1' or 'F1' → use VARCHAR(10), not INT

Now data types. You only need five. INT for IDs and counts. VARCHAR fifty for any text, like names and cities. DECIMAL ten comma two for money, like salary, fare and balance. Use DECIMAL four comma two for C G P A, and decimal two comma one for a rating like four point five. DATE for dates, written year dash

> month dash day inside quotes. One trap: if an ID has letters, like C1 for a course or F1 for a flight, it must be VARCHAR, not INT.

The 6 constraints — what each means

- PRIMARY KEY → unique + not null, identifies each row
- NOT NULL → value is compulsory
- UNIQUE → no duplicates (NULL allowed)
- CHECK (condition) → value must satisfy a rule
- DEFAULT value → used when you don't give a value
- FOREIGN KEY ... REFERENCES parent(col) → value must exist in parent table

> Constraints are rules on columns. Primary key means unique and not null, it identifies each row. Not null means the value is compulsory. Unique means no duplicates. Check, with a condition in brackets, means every value must satisfy that rule. Default gives a value automatically when you don't supply one. And foreign key references a parent table, which means the value must already exist in the parent table. Remember these one-line meanings, because the examiner may ask you to explain them in the viva.

RECIPE · CREATE TABLE template

1. Name, type, then constraint
2. Comma after every column except the last
3. FK lines go at the end

```
CREATE TABLE TableName (
    IDcol     INT PRIMARY KEY,
    NameCol   VARCHAR(50) NOT NULL,
    UniqueCol VARCHAR(50) UNIQUE,
    NumCol    DECIMAL(10,2) CHECK (NumCol > 20000),
    PriceCol  DECIMAL(6,2) DEFAULT 150,
    FKcol     INT,
    FOREIGN KEY (FKcol) REFERENCES Parent(ParentID)
);
```

Here is the master template. Write CREATE TABLE, the table name, and open a bracket. Then each column is: name, then type, then the constraint. Put a comma after every line except the last one. The check constraint is written as the word CHECK followed by a condition in brackets. Default is the word DEFAULT followed by the value. And foreign key lines go at the very end: FOREIGN KEY, the column in brackets, REFERENCES, the parent table, and the parent's column in brackets. Then close the bracket and add a semicolon. Every create-table question is just this template with different names.

EXAMPLE · Student table with all constraints

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name      VARCHAR(50) NOT NULL UNIQUE,
    Dept      VARCHAR(20) NOT NULL,
    City      VARCHAR(30),
    CGPA      DECIMAL(4,2) CHECK (CGPA BETWEEN 0 AND
10)
);

INSERT INTO Student VALUES (101, 'Arun',    'CSE',
'Chennai', 8.50);
INSERT INTO Student VALUES (102, 'Priya',   'ECE',
'Madurai', 9.10);
INSERT INTO Student VALUES (103, 'Karthik', 'CSE',
```

```
'Chennai', 7.20);
SELECT * FROM Student;
```

Let's apply it to the exam question: create the Student table with primary key, not null, unique, and check C G P A between zero and ten. Student I D is the primary key. Name is not null and unique. Dept is not null. And C G P A has CHECK, C G P A BETWEEN zero AND ten. Notice that between includes both ends. Then three inserts, with text in quotes and numbers without quotes, and finally select star to show the table.

EXAMPLE · CHECK, DEFAULT and NOT NULL variants

```
-- Salary must be above 20000
Salary DECIMAL(10,2) CHECK (Salary > 20000)

-- Balance at least 1000
Balance DECIMAL(12,2) CHECK (Balance >= 1000)

-- Movie price defaults to 150
TicketPrice DECIMAL(6,2) DEFAULT 150
INSERT INTO Movie (MovieID, MovieName, Genre) VALUES
(2, 'Jailer', 'Action');
--   ↑ column list without TicketPrice → it becomes
150

-- Specialization compulsory
Specialization VARCHAR(40) NOT NULL

-- TrainName must not repeat
TrainName VARCHAR(50) UNIQUE
```

Your paper has many small variations, and they're all one line. Salary greater than twenty thousand: CHECK Salary greater than twenty thousand. Balance at least one thousand: CHECK Balance greater than or equal to one thousand. Ticket price default one fifty:

DEFAULT one fifty. To prove the default works, insert with a column list that leaves ticket price out, and it automatically becomes one fifty. Specialization not null, and train name unique. Just read the question, find the column, and attach the right word.

RECIPE · Foreign keys (parent → child)

1. FK column type = parent's PK type
2. Parent rows must exist first

```
-- 1. Parent first
CREATE TABLE Course (
    CourseID   VARCHAR(10) PRIMARY KEY,
    CourseName VARCHAR(50) NOT NULL,
    Credits    INT
);
-- 2. Child with FOREIGN KEY lines at the end
CREATE TABLE Enrollment (
    EnrollID  INT PRIMARY KEY,
    StudentID INT,
    CourseID  VARCHAR(10),
    Marks     INT,
    FOREIGN KEY (StudentID) REFERENCES
Student(StudentID),
    FOREIGN KEY (CourseID)  REFERENCES
Course(CourseID)
);
INSERT INTO Course VALUES ('C1', 'DBMS', 4);
INSERT INTO Enrollment VALUES (1, 101, 'C1', 85);
```

Now foreign keys. The recipe is: parent tables first, then the child. A table that has FK in the schema, like Enrollment with student I D and course I D, is the child. The foreign key column must have exactly the same data type as the parent's primary key. So course I D is VARCHAR in both. At the end of the child table, add one FOREIGN KEY line for each FK. And when inserting, insert the course and student first, then the

enrollment. That's exactly the question: insert an enrollment for student one oh one and course C1.

RECIPE · Add a FK later with ALTER TABLE

```
ALTER TABLE Child
ADD CONSTRAINT fk_name
FOREIGN KEY (col) REFERENCES Parent(col);

-- Exam: FK from Employee.DeptID to Department.DeptID
ALTER TABLE Employee
ADD CONSTRAINT fk_dept
FOREIGN KEY (DeptID) REFERENCES Department(DeptID);

-- "Try inserting DeptID 99 and explain why it fails"
INSERT INTO Employee VALUES (205, 'Vijay', 99, 30000,
'2022-02-02');
-- ERROR 1452: Cannot add or update a child row:
--             a foreign key constraint fails
```

When the question says add a foreign key, it means the table already exists, so you use ALTER TABLE. ALTER TABLE child, ADD CONSTRAINT, any name like f k underscore dept, then the same FOREIGN KEY REFERENCES part. Your paper also asks: try inserting an employee with department ninety nine and explain why it fails. The answer: it fails with error fourteen fifty two, because department ninety nine does not exist in the parent table Department. That is called referential integrity. Write that sentence as a comment and you get full marks.

ON DELETE CASCADE / ON UPDATE CASCADE

```
CREATE TABLE Booking (
    BookingID   INT PRIMARY KEY,
    PassengerID INT,
    TrainID     INT,
    SeatsBooked INT,
```

```
    Fare        DECIMAL(8,2),
    FOREIGN KEY (PassengerID) REFERENCES
Passenger(PassengerID),
    FOREIGN KEY (TrainID) REFERENCES Train(TrainID)
ON DELETE CASCADE
);
DELETE FROM Train WHERE TrainID = 2;
SELECT * FROM Booking;    -- bookings of train 2 are
gone automatically

-- ON UPDATE CASCADE: change parent ID → child rows
follow
FOREIGN KEY (RestaurantID) REFERENCES
Restaurant(RestaurantID) ON UPDATE CASCADE
```

Cascade questions. Just add ON DELETE CASCADE, or ON UPDATE CASCADE, at the end of the foreign key line. Explain it like this. On delete cascade: when a train is deleted from the Train table, all bookings for that train are deleted automatically from Booking. On update cascade: when a restaurant I D changes in the parent, the same I D is updated automatically in the food order table. To demonstrate, delete or update the parent row, then select star from the child.

RECIPE · INSERT, UPDATE, DELETE

1. NEVER forget WHERE on UPDATE/DELETE
2. +10% → * 1.10 · +5% → * 1.05

```
-- INSERT (all columns, in table order)
INSERT INTO Table VALUES (v1, 'text', v3);
-- INSERT several rows at once
INSERT INTO Train VALUES (1, 'Vaigai Express',
'Chennai', 'Madurai'),
                         (2, 'Kovai Express',
'Chennai', 'Coimbatore');
-- UPDATE: SET what changes, WHERE which rows
UPDATE Employee SET Salary = 75000 WHERE EmpName =
'Meena';
UPDATE Flight   SET TicketPrice = TicketPrice + 500
```

```
WHERE FlightID = 'F1';
UPDATE Employee SET Salary = Salary * 1.10;        --
+10% for everyone
-- DELETE: WHERE which rows
DELETE FROM Train WHERE Destination = 'Coimbatore';
```

Now the three commands that change data. Insert into table values, with values in the same order as the columns. You can insert several rows by separating brackets with commas. Update table SET column equals new value WHERE condition. To add to an existing value, write column equals column plus five hundred. For a ten percent raise, multiply by one point one zero, and for five percent, one point zero five. Delete from table where condition. The most important point: never forget the WHERE clause on update and delete, or you'll change every row in the table.

Part 1 · Quick recap

- CREATE TABLE name (col TYPE constraint, … , FOREIGN KEY …);
- CHECK (cond) DEFAULT v NOT NULL UNIQUE PRIMARY KEY
- ALTER TABLE child ADD CONSTRAINT fk FOREIGN KEY (c) REFERENCES p(c);
- FK fails when parent value doesn't exist (referential integrity)
- ON DELETE CASCADE / ON UPDATE CASCADE → child follows parent
- INSERT INTO t VALUES (…); UPDATE t SET c=v WHERE …; DELETE FROM t WHERE …;

Quick recap of part one. Create table with name, type, constraint, and foreign key lines at the end. The five

column constraints: check, default, not null, unique, primary key. Alter table add constraint for a foreign key added later. A foreign key insert fails when the parent value doesn't exist. Cascade makes child rows follow the parent. And insert, update with set and where, delete with where. Pause here if you need to, and in part two we'll write select queries.

Part 2 · SELECT Queries, GROUP BY, HAVING & Subqueries

Part two: select queries. These are the most common questions in your paper, and they all follow one fixed order. Once you know the order, you just fill in the blanks.

THE ORDER — memorise this line

```
SELECT    columns
FROM      table
WHERE     row condition
GROUP BY  column
HAVING    group condition
ORDER BY  column ASC | DESC;

-- Memory trick:  "Some Funny Wizards Get Hungry
Often"
--                 SELECT FROM WHERE GROUP HAVING
ORDER
```

This is the most important slide in the whole course. Select columns, from table, where row condition, group by column, having group condition, order by column. You can skip any part you don't need, but the order never changes. A memory trick: Some Funny Wizards Get Hungry Often. Select, from, where, group by, having, order by.

WHERE conditions + ORDER BY

```
SELECT * FROM Student WHERE City = 'Chennai';
SELECT * FROM Student WHERE Dept = 'CSE' AND CGPA >
8;
```

```sql
SELECT * FROM Employee WHERE Salary BETWEEN 30000 AND
60000;
SELECT * FROM Student WHERE City IN ('Chennai',
'Madurai');
SELECT * FROM Student WHERE Name LIKE 'A%';     --
starts with A
SELECT * FROM Doctor WHERE Specialization IS NULL;

SELECT * FROM Flight ORDER BY TicketPrice DESC;  --
highest first
SELECT * FROM Student ORDER BY Name;             --
ASC is default
```

> Where filters rows. Equals for exact match, like City equals Chennai. Combine with AND or OR. Between for a range. IN for a list of values. LIKE with percent for patterns, so A percent means names starting with A. And IS NULL, never equals null. Order by sorts the result. DESC means highest first, and ASC, ascending, is the default. The exam question: display flights ordered by ticket price in descending order, is exactly that one line.

Aggregate functions

1. count → COUNT
2. total → SUM
3. average → AVG
4. highest → MAX · lowest → MIN

```sql
SELECT COUNT(*)   FROM Employee;            -- how
many rows
SELECT SUM(Fare)  FROM Booking;             -- total
SELECT AVG(CGPA)  FROM Student WHERE Dept = 'CSE';
SELECT MAX(Salary), MIN(Salary) FROM Employee;

-- give the result a nice name with AS
SELECT AVG(CGPA) AS Avg_CSE_CGPA FROM Student WHERE
Dept = 'CSE';
```

Aggregate functions squash many rows into one number. Read the question and translate the words. Number of, or how many, means COUNT. Total means SUM. Average means AVG. Highest means MAX and lowest means MIN. Use AS to give the column a nice name. So, find the average C G P A of C S E students, is select A V G of C G P A from student where dept equals C S E.

What GROUP BY actually does

1. Rows with the SAME value go into one pile
2. COUNT / SUM / MAX / AVG run on EACH pile
3. Output = ONE row per pile

```
-- Employee table (sorted by DeptID so you can see
the piles)
-- EmpName  DeptID  Salary
-- Ravi       1     45000    <- pile for Dept 1  (1
person)
-- ------------------------
-- Meena      2     75000    \
-- Suresh     2     52000     > pile for Dept 2  (3
people)
-- Kavya      2     61000    /
-- ------------------------
-- Anil       3     35000    <- pile for Dept 3  (1
person)

SELECT DeptID, COUNT(*), MAX(Salary), SUM(Salary)
FROM Employee
GROUP BY DeptID;
-- DeptID | COUNT(*) | MAX(Salary) | SUM(Salary)
--    1   |    1     |   45000     |   45000
--    2   |    3     |   75000     |  188000
--    3   |    1     |   35000     |   35000
```

Let's understand what group by really does. Look at the employee table. Group by dept I D takes all the

rows and puts rows with the same dept I D into one pile. Ravi is alone in department one. Meena, Suresh and Kavya go into the pile for department two. Anil is alone in department three. So we get three piles. Then the aggregate functions run separately on each pile. Count star counts the people in each pile: one, three, one. Max salary finds the highest in each pile. Sum adds up each pile, so department two is seventy five thousand plus fifty two thousand plus sixty one thousand, which is one lakh eighty eight thousand. And the output has exactly one row per pile. Five employees in, three rows out, because there are three departments. That's all group by is: make piles, then calculate per pile.

Without GROUP BY vs with GROUP BY

1. No GROUP BY → 1 answer for the whole table
2. GROUP BY col → 1 answer for each value of col
3. "highest salary" vs "highest salary in EACH dept"

```
-- WITHOUT GROUP BY: the whole table is ONE pile
SELECT COUNT(*), MAX(Salary) FROM Employee;
-- COUNT(*) | MAX(Salary)
--    5     |   75000          <- one answer for
everyone

-- WITH GROUP BY: one pile per department
SELECT DeptID, COUNT(*), MAX(Salary) FROM Employee
GROUP BY DeptID;
-- DeptID | COUNT(*) | MAX(Salary)
--    1   |    1     |   45000
--    2   |    3     |   75000
--    3   |    1     |   35000   <- one answer per
department
```

Compare the two. Without group by, the whole table is treated as one big pile, so count star gives five and

max salary gives seventy five thousand, one answer for everyone. With group by dept I D, you get one answer per department. So, if the question says highest salary, you don't need group by. If it says highest salary in each department, you need group by dept I D. The word each is your signal.

The GROUP BY rule: what can go in SELECT?

1. SELECT only: the GROUP BY column(s) + aggregates
2. Every other column → error (ONLY_FULL_GROUP_BY)
3. Same column in SELECT and GROUP BY

```
-- CORRECT: grouped column + aggregates
SELECT DeptID, COUNT(*), AVG(Salary)
FROM Employee
GROUP BY DeptID;

-- WRONG: EmpName is not grouped
SELECT DeptID, EmpName, COUNT(*)
FROM Employee
GROUP BY DeptID;
-- Dept 2 has Meena, Suresh AND Kavya... which ONE
name
-- should go in Dept 2's single row?
-- → ERROR 1055: EmpName isn't in GROUP BY

-- Group by MORE than one column = smaller piles
SELECT DeptID, YEAR(HireDate), COUNT(*)
FROM Employee
GROUP BY DeptID, YEAR(HireDate);
```

One rule you must follow. When you use group by, the select list can only contain the column you grouped by, plus aggregate functions like count, sum and average. Why? Department two's pile has three people, Meena, Suresh and Kavya, but the output has

only one row for department two. So which name would MySQL print? It can't decide, so MySQL eight gives an error. The easy way to remember: whatever column is in group by, put the same column in select, and everything else must be inside count, sum, average, max or min. You can also group by two columns, which simply makes smaller piles.

RECIPE · GROUP BY ("for each ...")

1. Spot the words "each" / "per" / "for every"
2. That column goes in SELECT and GROUP BY

```
SELECT group_col, AGG(col) AS name
FROM   table
GROUP BY group_col;

-- Highest salary in each department
SELECT DeptID, MAX(Salary) AS HighestSalary
FROM Employee
GROUP BY DeptID;

-- Total fare collected for each train
SELECT TrainID, SUM(Fare) AS TotalFare
FROM Booking
GROUP BY TrainID;
```

Group by. Whenever you see the words for each, per, or in every, you need group by. The recipe: the column after the word each goes in both select and group by, and then you add the aggregate. Highest salary in each department: select dept I D, max salary, from employee, group by dept I D. Total fare for each train: select train I D, sum of fare, group by train I D. Same shape every time.

RECIPE · HAVING (condition on a group)

1. WHERE filters ROWS (before grouping)
2. HAVING filters GROUPS (after grouping)
3. Condition uses COUNT/SUM/AVG → HAVING

```sql
-- Departments having more than one employee
SELECT DeptID, COUNT(*) AS EmpCount
FROM Employee
GROUP BY DeptID
HAVING COUNT(*) > 1;

-- Movies whose average seats booked > 3
SELECT MovieID, AVG(SeatsBooked) AS AvgSeats
FROM Booking
GROUP BY MovieID
HAVING AVG(SeatsBooked) > 3;
```

Having. Where filters individual rows before grouping. Having filters groups after grouping. The simple test: if your condition contains count, sum, average, max or min, it must go in HAVING. Departments having more than one employee: group by dept I D, having count star greater than one. Movies with average seats greater than three: group by movie I D, having average seats booked greater than three.

WHERE vs HAVING: the order MySQL runs it

1. WHERE filters ROWS (before piles)
2. HAVING filters PILES (after GROUP BY)
3. COUNT/SUM/AVG in a condition → HAVING

```sql
SELECT DeptID, COUNT(*) AS EmpCount
FROM Employee            -- 1. take the table
WHERE Salary > 40000     -- 2. remove ROWS: Anil
(35000) goes
GROUP BY DeptID          -- 3. make piles: D1 = Ravi
                         --    D2 = Meena, Suresh,
```

```
Kavya
HAVING COUNT(*) > 1      -- 4. remove PILES: D1 has 1
→ goes
ORDER BY DeptID;         -- 5. sort

-- Result:   DeptID | EmpCount
--              2   |    3

-- WHERE COUNT(*) > 1  → ERROR (no piles yet at step
2)
```

Here's the order MySQL actually runs a grouped query, and it makes where versus having very clear. Step one, from: pick the employee table. Step two, where salary greater than forty thousand removes individual rows, so Anil with thirty five thousand is gone before any grouping. Step three, group by makes the piles: department one has only Ravi, department two has Meena, Suresh and Kavya. Step four, having count star greater than one removes whole piles, so department one is removed because it has only one person. Step five, select and order by. The result is department two with three employees. And notice: you cannot write where count star greater than one, because at the where step the piles don't exist yet. That's why conditions with count, sum or average always go in having.

WHERE + GROUP BY together

```
-- For each city: total orders, average price, total
quantity
-- where rating is at least 4.2
SELECT r.City,
       COUNT(f.OrderID) AS TotalOrders,
       AVG(f.Price)     AS AvgPrice,
       SUM(f.Quantity)  AS TotalQty
FROM FoodOrder f
JOIN Restaurant r ON f.RestaurantID = r.RestaurantID
```

```sql
WHERE f.Rating >= 4.2
GROUP BY r.City;

-- Count appointments handled by each doctor
SELECT DoctorID, COUNT(*) AS TotalAppointments
FROM Appointment
GROUP BY DoctorID;
```

Some questions combine everything. For each city, display total orders, average price and total quantity, where rating is at least four point two. Rating is a normal row condition, so it goes in WHERE. For each city means group by city. City lives in the Restaurant table, so we join it in. We'll study joins in part three. And the count of appointments per doctor is just group by doctor I D with count star.

RECIPE · Subquery: compare with average / max

1. You can't write WHERE CGPA > AVG(CGPA)
2. Put the AVG in its own SELECT in brackets

```sql
SELECT cols FROM table
WHERE col > (SELECT AVG(col) FROM table);

-- Students whose CGPA > average CGPA
SELECT Name FROM Student
WHERE CGPA > (SELECT AVG(CGPA) FROM Student);

-- Employees earning more than average
SELECT EmpID, EmpName, Salary FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);

-- Accounts with balance = maximum balance
SELECT * FROM Account
WHERE Balance = (SELECT MAX(Balance) FROM Account);
```

Subqueries. This exact question appears in almost every set: find rows greater than the average. You cannot write where C G P A greater than average C G

P A directly. Instead, compute the average in its own little select, inside brackets. Where C G P A is greater than, bracket, select average C G P A from student, close bracket. The inner query runs first and gives one number. Swap AVG for MAX to find the account with the maximum balance. Learn this template, it is guaranteed marks.

RECIPE · IN / NOT IN ("at least one" / "never")

1. "at least one" → IN
2. "never" / "no" → NOT IN
3. Linking column = the common ID

```
-- Passengers who made at least one booking
SELECT Name FROM Passenger
WHERE PassengerID IN (SELECT PassengerID FROM
Booking);

-- Doctors who have at least one appointment
SELECT * FROM Doctor
WHERE DoctorID IN (SELECT DoctorID FROM Appointment);

-- Restaurants with at least one order rated 5
SELECT * FROM Restaurant
WHERE RestaurantID IN (SELECT RestaurantID FROM
FoodOrder WHERE Rating = 5);

-- Restaurants that NEVER received any order
SELECT * FROM Restaurant
WHERE RestaurantID NOT IN (SELECT RestaurantID FROM
FoodOrder);
```

Second subquery pattern. At least one means IN. Never, or have not, means NOT IN. The recipe: select from the main table, where its I D is IN, bracket, select that same I D from the other table. Passengers who made at least one booking: passenger I D in select passenger I D from booking. Doctors with at least one

appointment: same shape. Restaurants that never received an order: just change IN to NOT IN. You can even add a where inside the brackets, like rating equals five.

Correlated subquery ("of that same ...")

1. Give the outer table an alias (b1)
2. Inner query matches on b2.col = b1.col
3. Runs once per outer row

```
-- Bookings whose fare > average fare of THAT SAME
train
SELECT * FROM Booking b1
WHERE Fare > (SELECT AVG(Fare)
              FROM Booking b2
              WHERE b2.TrainID = b1.TrainID);
```

The trickiest one: bookings whose fare is greater than the average fare of that same train. The phrase that same tells you it's a correlated subquery. Give the outer table a nickname, b1, and the inner one b2. Inside the brackets, add where b2 dot train I D equals b1 dot train I D. That way, for each booking, the average is calculated only for its own train. Don't worry, it's the same as the average template, plus one extra where line.

Part 2 · Word → SQL dictionary

- "for each / per" → GROUP BY that column
- "more than N in a group" → HAVING COUNT(*) > N
- "greater than average" → WHERE col > (SELECT AVG(col) FROM t)

- "equal to maximum" → WHERE col = (SELECT MAX(col) FROM t)
- "at least one" → WHERE id IN (SELECT id FROM other)
- "never / no" → WHERE id NOT IN (SELECT id FROM other)
- "of that same X" → correlated subquery
- "descending / highest first" → ORDER BY col DESC

> Here's your word to S Q L dictionary. For each: group by. More than N in a group: having count. Greater than average: subquery with AVG. Equal to maximum: subquery with MAX. At least one: IN. Never: NOT IN. Of that same: correlated subquery. Descending: order by desc. Take a screenshot of this slide, it decodes almost every select question in your paper.

Part 3 · JOINs

Part three: joins. Any time the question asks for columns that live in different tables, like patient name and doctor name, you need a join. Let's see the recipe.

RECIPE · INNER JOIN (2 tables)

1. 1 · Which tables hold the columns?
2. 2 · Give each a short alias
3. 3 · ON = the common ID column

```
SELECT a.col, b.col
FROM TableA a
JOIN TableB b ON a.commonID = b.commonID;

-- Employee name with department name
SELECT e.EmpName, e.Salary, d.DeptName
FROM Employee e
JOIN Department d ON e.DeptID = d.DeptID;
```

The join recipe has three steps. One: find which tables hold the columns you need. Two: give each table a short nickname, like e for employee and d for department. Three: join them ON the common I D column, the one that appears in both tables. So, employee name with department name: from employee e, join department d, on e dot dept I D equals d dot dept I D. Then use the nicknames in the select list.

RECIPE · 3-table JOIN (via the middle table)

1. Middle table = the one with 2 FKs
2. Booking, Ticket, Appointment, Enrollment, FoodOrder
3. One JOIN ... ON per FK

```
-- Patient names and doctor names for all
appointments
SELECT p.PatientName, d.DoctorName
FROM Appointment a
JOIN Patient p ON a.PatientID = p.PatientID
JOIN Doctor  d ON a.DoctorID  = d.DoctorID;

-- Student names and course names for all enrolled
students
SELECT s.Name, c.CourseName
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Course     c ON e.CourseID  = c.CourseID;
```

> Most of your schemas have three tables, and the middle one has two foreign keys. Appointment links patient and doctor. Enrollment links student and course. Booking, ticket and food order work the same way. The recipe: start from the middle table, then add one join for each foreign key. Appointment a, join patient p on patient I D, join doctor d on doctor I D. That's it. Two join lines for three tables.

LEFT JOIN (include rows with no match)

1. "include ... who have not" → LEFT JOIN
2. Table you want ALL rows of goes FIRST

```
-- All passengers and their bookings,
-- INCLUDING passengers who have not booked
SELECT p.PassengerID, p.Name, b.BookingID, b.TrainID,
b.Fare
```

```
FROM Passenger p
LEFT JOIN Booking b ON p.PassengerID = b.PassengerID;
--  passengers with no booking show NULL in booking
columns
```

Left join. When the question says include passengers who have not booked, a normal join would drop them. A left join keeps every row of the first table, and fills null where there is no match. The rule: the table you want all rows of goes first, on the left. So from passenger p, left join booking b. Passengers without a booking appear with null in the booking columns.

FULL OUTER JOIN in MySQL (UNION trick)

1. MySQL has NO FULL OUTER JOIN keyword
2. LEFT JOIN UNION RIGHT JOIN
3. Same column list in both halves

```
-- Every booking and every movie, matched where
possible,
-- blanks (NULL) for unmatched rows
SELECT b.BookingID, b.SeatsBooked, m.MovieID,
m.MovieName
FROM Booking b LEFT JOIN Movie m ON b.MovieID =
m.MovieID
UNION
SELECT b.BookingID, b.SeatsBooked, m.MovieID,
m.MovieName
FROM Booking b RIGHT JOIN Movie m ON b.MovieID =
m.MovieID;
```

Full outer join. The question says: show every booking and every movie, matched where possible, with blanks for unmatched rows. That wording means full outer join. But MySQL does not support the full outer join keyword! The trick is: write a left join, then the word UNION, then the same query as a right join. Union

> combines both and removes duplicates. Mention this in the viva, examiners love it.

CROSS JOIN and SELF JOIN

1. "all possible combinations" → CROSS JOIN
2. "pairs ... same" → SELF JOIN (table joined to itself)

```sql
-- All possible passenger–flight combinations
SELECT p.Name, f.FlightName
FROM Passenger p
CROSS JOIN Flight f;          -- rows = passengers × flights

-- Pairs of employees who work in the same department
SELECT e1.EmpName AS Employee1, e2.EmpName AS Employee2, e1.DeptID
FROM Employee e1
JOIN Employee e2
  ON e1.DeptID = e2.DeptID
 AND e1.EmpID < e2.EmpID;     -- avoids (A,A) and duplicate (B,A)
```

> Two more. All possible combinations means cross join. It pairs every passenger with every flight, so the number of rows is passengers times flights. No ON clause needed. Pairs of employees in the same department means self join: the employee table joined with itself, using two nicknames, e1 and e2. Join on same dept I D, and add e1 dot emp I D less than e2 dot emp I D, so you don't pair someone with themselves or list the same pair twice.

Part 3 · Which JOIN?

- Columns from 2–3 tables, only matches → (INNER) JOIN ... ON

- "include those who have not ..." → LEFT JOIN (keep-all table first)
- "every A and every B, blanks for unmatched" → LEFT JOIN UNION RIGHT JOIN
- "all possible combinations" → CROSS JOIN
- "pairs ... in the same ..." → SELF JOIN with e1.id < e2.id
- 3 tables → start from the middle table (the one with 2 FKs)

> Recap. Normal matches: inner join on the common I D. Include those who have not: left join. Every A and every B with blanks: left join union right join. All combinations: cross join. Pairs in the same group: self join. And for three tables, start from the middle table. That's joins done. Next, transactions and permissions.

Part 4 · Transactions (COMMIT/ROLLBACK/SAVEPOINT) + GRANT/REVOKE

> Part four. Transactions are the biggest topic in your question papers. Nearly a third of the questions say commit, rollback or savepoint. The good news is, they're all one recipe.

The 4 words

- START TRANSACTION; → begin a group of changes
- COMMIT; → save permanently (cannot be undone)
- ROLLBACK; → undo everything since START TRANSACTION
- SAVEPOINT name; → a checkpoint in the middle
- ROLLBACK TO name; → undo only what came after that checkpoint
- First run SET autocommit = 0; (MySQL saves every statement by default)

> Four words. Start transaction begins a group of changes. Commit saves them permanently. Rollback undoes everything since start transaction. Savepoint, with a name, creates a checkpoint in the middle. And rollback to that name undoes only the changes after the checkpoint. One important detail: MySQL automatically commits every statement by default, so

> first run set autocommit equals zero. Otherwise rollback has nothing to undo.

RECIPE · "... Use COMMIT"

1. START TRANSACTION
2. do the changes the question lists
3. COMMIT
4. SELECT to show

```
SET autocommit = 0;
START TRANSACTION;
INSERT INTO Student VALUES (104, 'Divya', 'IT',
'Coimbatore', 8.80);
INSERT INTO Enrollment VALUES (3, 104, 'C1', 90);
COMMIT;
SELECT * FROM Enrollment;      -- both rows saved
permanently
```

> Recipe one: the question says do something, then use commit. Write start transaction, then exactly the changes the question lists, then commit, then select star to show the result. Insert a new student and enroll them in a course, use commit: start transaction, two inserts, commit. Done.

RECIPE · "... Use ROLLBACK to undo / restore"

1. START TRANSACTION
2. change
3. SELECT (shows change)
4. ROLLBACK
5. SELECT (shows original)

```
START TRANSACTION;
DELETE FROM Enrollment WHERE EnrollID = 2;
```

```
SELECT * FROM Enrollment;       -- row 2 missing
ROLLBACK;
SELECT * FROM Enrollment;       -- row 2 is back!

START TRANSACTION;
UPDATE Student SET CGPA = 9.50 WHERE StudentID = 103;
INSERT INTO Enrollment VALUES (4, 103, 'C2', 70);
ROLLBACK;                       -- BOTH changes undone
```

> Recipe two: delete or update something and use rollback to restore it. Start transaction, make the change, select to show it happened, then rollback, then select again to show it's back. Showing the select before and after proves that you understand. Rollback undoes everything since start transaction, so if you did an update and an insert, both are undone.

RECIPE · SAVEPOINT + ROLLBACK TO

1. Step A
2. SAVEPOINT sp1
3. Step B
4. ROLLBACK TO sp1 → only B undone
5. COMMIT → A kept

```
START TRANSACTION;
INSERT INTO Student VALUES (105, 'Rahul', 'MECH',
'Salem', 7.90);  -- step A
SAVEPOINT sp1;
INSERT INTO Enrollment VALUES (5, 105, 'C1', 65);
-- step B
ROLLBACK TO sp1;                -- undoes ONLY step B
COMMIT;                         -- saves step A

SELECT * FROM Student    WHERE StudentID = 105;   --
Rahul is there
SELECT * FROM Enrollment WHERE StudentID = 105;   --
empty
```

Recipe three, the savepoint. Questions say: do A, create a savepoint, do B, roll back to the savepoint. Write it in exactly that order. Start transaction, step A, savepoint s p 1, step B, rollback to s p 1. Only step B is undone, step A stays. Then commit to keep A. And to verify which changes remain, select both tables. Rahul the student remains, his enrollment is gone.

SAVEPOINT variations from your papers

```sql
-- +10%, savepoint, +5% more, roll back to first
savepoint
START TRANSACTION;
UPDATE Employee SET Salary = Salary * 1.10;
SAVEPOINT after_10;
UPDATE Employee SET Salary = Salary * 1.05;
ROLLBACK TO after_10;          -- only the +10% stays
COMMIT;

-- Book two tickets, roll back only the second
START TRANSACTION;
INSERT INTO Ticket VALUES (3, 1, 'F1', '1A');
SAVEPOINT after_first;
INSERT INTO Ticket VALUES (4, 2, 'F2', '2B');
ROLLBACK TO after_first;
COMMIT;
```

Here are two variations from your papers. Increase salary ten percent, savepoint, increase another five percent, roll back to the savepoint: only the ten percent stays. Book two tickets, savepoint after the first, roll back the second. Same shape both times: first change, savepoint, second change, rollback to. You'll see this with employees, bookings, orders, appointments, all of them. Only the table changes.

Bank transfer (all-or-nothing)

```
START TRANSACTION;
UPDATE Account SET Balance = Balance - 10000 WHERE
AccountNo = 1002;
UPDATE Account SET Balance = Balance + 10000 WHERE
AccountNo = 1003;
INSERT INTO `Transaction` VALUES (3, 1002, 10000,
'Debit');
INSERT INTO `Transaction` VALUES (4, 1003, 10000,
'Credit');
COMMIT;
-- If ANY statement shows an error → type  ROLLBACK;
instead of COMMIT
-- Tip: Transaction is a keyword → wrap the table
name in `backticks`
```

> The bank transfer question. Transfer ten thousand from one account to another and record both transactions, roll back if any operation fails. Subtract from the first account, add to the second, insert two transaction records, a debit and a credit, then commit. Explain that if any statement fails, you type rollback instead, so money never disappears. This is called atomicity, all or nothing. Also, transaction is a keyword, so wrap that table name in backticks.

RECIPE · GRANT and REVOKE

1. GRANT ... TO
2. REVOKE ... FROM
3. "all privileges" → ALL PRIVILEGES

```
-- create the user first (MySQL 8 needs this)
CREATE USER 'student_user'@'localhost' IDENTIFIED BY
'pass123';

GRANT  SELECT, INSERT  ON db.Table  TO
'user'@'localhost';
REVOKE INSERT          ON db.Table  FROM
```

```
'user'@'localhost';

GRANT SELECT ON college.Student TO
'student_user'@'localhost';
GRANT SELECT, INSERT, UPDATE ON college.Student TO
'faculty_user'@'localhost';
REVOKE INSERT ON college.Enrollment FROM
'faculty_user'@'localhost';
GRANT ALL PRIVILEGES ON college.Course TO
'course_admin'@'localhost';

SHOW GRANTS FOR 'faculty_user'@'localhost';
```

> Grant and revoke. First create the user: create user, name in quotes, at localhost, identified by a password. Then: GRANT, the permissions separated by commas, ON database dot table, TO the user. Revoke is the same, but with FROM instead of TO. Remember: grant to, revoke from. All privileges is written ALL PRIVILEGES. To prove it worked, show grants for the user. Every grant question in your paper is just this pattern with different names.

Part 4 · Recap

- SET autocommit = 0; then START TRANSACTION;
- change → COMMIT (permanent)
- change → SELECT → ROLLBACK → SELECT (restored)
- A → SAVEPOINT sp → B → ROLLBACK TO sp → COMMIT (A kept, B undone)
- GRANT perms ON db.table TO 'u'@'localhost';
- REVOKE perms ON db.table FROM 'u'@'localhost';

> Recap. Set autocommit zero, start transaction. Change then commit makes it permanent. Change, select,

rollback, select shows it's restored. A, savepoint, B, rollback to savepoint, commit keeps A and undoes B. Grant to, revoke from. Great job, you've finished the biggest part of the exam.

Part 5 · Views & Indexes

> Part five, views and indexes. This is the easiest part. Every set has one of these, and the answer is just a join with one extra line on top.

RECIPE · CREATE VIEW

1. A view = a saved SELECT (virtual table)
2. Write the JOIN, then put CREATE VIEW ... AS on top

```
CREATE VIEW View_Name AS
SELECT ...                  -- any SELECT / JOIN you
already know
FROM ...
JOIN ... ON ...;

SELECT * FROM View_Name;    -- use it like a table
```

> A view is a saved select query that behaves like a virtual table. The recipe is: write the join exactly like in part three, then add CREATE VIEW, the name, AS, on top. Then select star from the view to show it works. In the viva, say: a view is a virtual table, it doesn't store data, it runs the query each time.

EXAMPLE · 3-table view + index

1. List columns in the order the question says
2. Middle table first, one JOIN per FK

```
CREATE VIEW Ticket_Details_View AS
SELECT t.TicketID, p.Name, f.FlightName, f.Source,
f.Destination, t.SeatNo
```

```
FROM Ticket t
JOIN Passenger p ON t.PassengerID = p.PassengerID
JOIN Flight    f ON t.FlightID    = f.FlightID;

SELECT * FROM Ticket_Details_View;

CREATE INDEX idx_flight_source ON Flight(Source);
SHOW INDEX FROM Flight;
```

> Here's a full exam answer. Create a view named ticket details view showing ticket I D, passenger name, flight name, source, destination and seat number. Start from the middle table, ticket, join passenger, join flight. Then the index: create index, the name the question gives you, ON the table, and the column in brackets. Show index from flight proves it exists.

All the view questions — same shape

```
CREATE VIEW Employee_Department_View AS
SELECT e.EmpID, e.EmpName, e.Salary, d.DeptName
FROM Employee e JOIN Department d ON e.DeptID =
d.DeptID;

CREATE VIEW Customer_Account_View AS
SELECT c.CustomerID, c.CustomerName, a.AccountNo,
a.AccountType, a.Balance
FROM Customer c JOIN Account a ON c.CustomerID =
a.CustomerID;

CREATE VIEW Student_Course_View AS
SELECT s.StudentID, s.Name, c.CourseName, e.Marks
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Course     c ON e.CourseID  = c.CourseID;
```

> Look how all the view questions are the same. Employee department view: a two table join. Customer account view: a two table join. Student course view: a three table join through enrollment. Booking details,

appointment details, movie booking and food order views all follow the three table version. Just copy the column list from the question.

RECIPE · CREATE INDEX

1. Index = faster searching on that column
2. Name hint: idx_table_column

```
CREATE INDEX idx_name ON Table(Column);

CREATE INDEX idx_student_dept          ON
Student(Dept);
CREATE INDEX idx_employee_salary       ON
Employee(Salary);
CREATE INDEX idx_train_destination     ON
Train(Destination);
CREATE INDEX idx_doctor_specialization ON
Doctor(Specialization);
CREATE INDEX idx_account_balance       ON
Account(Balance);
CREATE INDEX idx_movie_genre           ON
Movie(Genre);

SHOW INDEX FROM Table;
DROP INDEX idx_name ON Table;
```

Indexes. Create index name, ON table, column in brackets. Notice the name always tells you the table and column: i d x underscore movie underscore genre means the Movie table, Genre column. So you can't go wrong. An index makes searching on that column faster, like the index at the back of a book. Show index from the table to verify.

Part 6 · Stored Procedures & Functions

Part six: stored procedures and functions. These look scary, but there are only seven questions in your papers, and they all use one skeleton.

Why DELIMITER?

- Inside BEGIN ... END we use ; for each statement
- MySQL would think the procedure ends at the first ;
- So: change the delimiter to // before, and back to ; after
- DELIMITER // ... END // DELIMITER ;
- Procedure → run with CALL name(...);
- Function → RETURNS a value, use inside SELECT name(...);

First, why delimiter. Inside a procedure we write statements ending with semicolons. But MySQL would think the whole procedure ends at the first semicolon. So we temporarily change the end symbol to double slash. Write delimiter double slash before, end double slash at the end, and delimiter semicolon to switch back. Procedures are run with CALL. Functions return a value and are used inside select.

RECIPE · Procedure skeleton

1. IN parameter = input value

2. No parameters → empty ( )

3. Body = normal SQL

```
DELIMITER //
CREATE PROCEDURE ProcName(IN p_id INT)
BEGIN
    any SQL statements;
END //
DELIMITER ;

CALL ProcName(101);
```

> The procedure skeleton. Delimiter double slash. Create procedure, a name, and in brackets the input parameters: the word IN, a name, and a type. Then BEGIN, your normal S Q L statements, END double slash, delimiter semicolon. Run it with call, the name, and the values. If there are no inputs, just write empty brackets.

The 4 procedures from your papers

```
DELIMITER //
CREATE PROCEDURE Welcome()
BEGIN
    SELECT 'Welcome to the DBMS Lab!' AS Message;
END //

CREATE PROCEDURE GetStudent(IN p_id INT)
BEGIN
    SELECT * FROM Student WHERE StudentID = p_id;
END //

CREATE PROCEDURE IncreaseMarks(IN p_id INT, IN p_inc
INT)
BEGIN
    UPDATE Student SET Marks = Marks + p_inc WHERE
StudentID = p_id;
END //

CREATE PROCEDURE DeleteStudent(IN p_id INT)
```

```
BEGIN
    DELETE FROM Student WHERE StudentID = p_id;
END //
DELIMITER ;
```

All four procedures in your papers. Welcome: select a message in quotes. Display student details by I D: select star where student I D equals p underscore i d. Increase marks of a specified student: update set marks equals marks plus p underscore inc. Delete a student: delete where I D equals p underscore i d. See how the body is just the select, update and delete you already know? The parameter simply replaces the fixed value.

Calling them (show the output!)

```
CALL Welcome();
CALL GetStudent(2);
CALL IncreaseMarks(1, 5);        -- student 1 gets +5
marks
SELECT * FROM Student;
CALL DeleteStudent(3);
SELECT * FROM Student;

-- Use this table for the student procedures
CREATE TABLE Student (StudentID INT PRIMARY KEY, Name
VARCHAR(50),
                      Dept VARCHAR(20), Marks INT);
INSERT INTO Student VALUES (1,'Arun','CSE',72),
(2,'Priya','IT',35), (3,'Kumar','ECE',58);
```

Always call them and show the result. Call welcome. Call get student two. Call increase marks one comma five, then select to show the marks went up. Call delete student three, then select. If your exam doesn't give you a student table with marks, create this small

one first: student I D, name, dept, marks, and insert three rows.

RECIPE · Function skeleton

1. No IN keyword for function params
2. RETURNS (with S) = type
3. RETURN (no S) = value
4. DETERMINISTIC = same input → same output

```
DELIMITER //
CREATE FUNCTION FuncName(a INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN expression;
END //
DELIMITER ;

SELECT FuncName(5);
```

The function skeleton. Create function, the name, parameters without the word IN. Then RETURNS, with an S, followed by the type of the answer. Then the word DETERMINISTIC, which means the same input always gives the same output. MySQL insists on it. Then begin, RETURN without an S, followed by the value, and end. Use it inside a select. Watch out: returns with an S for the type, return without an S for the value.

The 3 functions from your papers

```
DELIMITER //
CREATE FUNCTION Square(n INT) RETURNS INT
DETERMINISTIC
BEGIN
    RETURN n * n;
```

```
END //

CREATE FUNCTION AddNumbers(a INT, b INT) RETURNS INT
DETERMINISTIC
BEGIN
    RETURN a + b;
END //

CREATE FUNCTION PassOrFail(m INT) RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN
    IF m >= 40 THEN
        RETURN 'PASS';
    ELSE
        RETURN 'FAIL';
    END IF;
END //
DELIMITER ;

SELECT Square(7);  SELECT AddNumbers(10, 20);
SELECT Name, Marks, PassOrFail(Marks) AS Result FROM
Student;
```

> The three functions. Square: return n times n. Add two numbers: two parameters, return a plus b. Pass or fail: it returns text, so returns VARCHAR ten. Inside, IF m greater than or equal to forty THEN return pass, ELSE return fail, END IF, with a semicolon. Then select square of seven gives forty nine, add numbers ten and twenty gives thirty, and pass or fail can run on every student's marks.

Part 6 · Recap

- DELIMITER // ... END // DELIMITER ;
- PROCEDURE: CREATE PROCEDURE p(IN x INT) BEGIN ... END // → CALL p(1);
- FUNCTION: CREATE FUNCTION f(x INT) RETURNS INT DETERMINISTIC BEGIN RETURN ...; END //

- Use function: SELECT f(5);
- IF cond THEN … ELSE … END IF;
- Errors? DROP PROCEDURE IF EXISTS p; DROP FUNCTION IF EXISTS f;

> Recap. Delimiter double slash at the start, end double slash, delimiter semicolon. Procedures take IN parameters and you call them. Functions use returns type, deterministic, and return value, and you select them. If you make a mistake and need to re-create one, first drop procedure if exists, or drop function if exists.

Part 7 · Triggers (INSERT / UPDATE / DELETE)

> Part seven: triggers. Your papers ask for insert, update and delete triggers on their own. One template covers all three.

What is a trigger?

- Code that runs AUTOMATICALLY when a row is inserted / updated / deleted
- BEFORE or AFTER the event
- NEW.col → the new value (INSERT, UPDATE)
- OLD.col → the old value (UPDATE, DELETE)
- Typical use: write an entry into a LOG (audit) table
- Plan: 1 main table + 1 log table + trigger + test

> A trigger is code that runs automatically when a row is inserted, updated or deleted, either before or after the event. Inside it, NEW dot column means the new value, and OLD dot column means the old value. Insert only has NEW. Delete only has OLD. Update has both. The usual exam answer writes a record into a log table. So the plan is: one main table, one log table, the trigger, and then a test.

Step 1 · Main table + log table

```
CREATE TABLE Account (
    AccountNo INT PRIMARY KEY,
    Name      VARCHAR(50),
    Balance   DECIMAL(10,2)
```

```
);
CREATE TABLE Account_Log (
    LogID      INT AUTO_INCREMENT PRIMARY KEY,
    AccountNo  INT,
    Action     VARCHAR(20),
    OldBalance DECIMAL(10,2),
    NewBalance DECIMAL(10,2),
    ActionTime DATETIME
);
```

Step one. A main table, here account, with account number, name and balance. And a log table with an auto increment I D, the account number, an action word, the old balance, the new balance, and the time. Auto increment means MySQL numbers the log rows by itself. You can use any table from your schema as the main table, Employee, Student, whatever the question gives you.

RECIPE · Trigger skeleton

1. AFTER / BEFORE
2. INSERT / UPDATE / DELETE
3. FOR EACH ROW (always)
4. Body uses NEW / OLD

```
DELIMITER //
CREATE TRIGGER trg_name
AFTER INSERT ON Table
FOR EACH ROW
BEGIN
    INSERT INTO Log_Table (...) VALUES ( NEW.col /
OLD.col, ... );
END //
DELIMITER ;
```

The trigger skeleton. Delimiter double slash. Create trigger, a name. Then AFTER or BEFORE, then INSERT,

UPDATE or DELETE, ON the table. Then always FOR EACH ROW. Begin, and inside, insert into the log table using NEW or OLD values. End double slash, delimiter semicolon.

INSERT · UPDATE · DELETE triggers

1. INSERT → NEW only
2. UPDATE → OLD and NEW
3. DELETE → OLD only

```
DELIMITER //
CREATE TRIGGER trg_after_insert AFTER INSERT ON
Account FOR EACH ROW
BEGIN
    INSERT INTO Account_Log (AccountNo, Action,
OldBalance, NewBalance, ActionTime)
    VALUES (NEW.AccountNo, 'INSERT', NULL,
NEW.Balance, NOW());
END //

CREATE TRIGGER trg_after_update AFTER UPDATE ON
Account FOR EACH ROW
BEGIN
    INSERT INTO Account_Log (AccountNo, Action,
OldBalance, NewBalance, ActionTime)
    VALUES (NEW.AccountNo, 'UPDATE', OLD.Balance,
NEW.Balance, NOW());
END //

CREATE TRIGGER trg_after_delete AFTER DELETE ON
Account FOR EACH ROW
BEGIN
    INSERT INTO Account_Log (AccountNo, Action,
OldBalance, NewBalance, ActionTime)
    VALUES (OLD.AccountNo, 'DELETE', OLD.Balance,
NULL, NOW());
END //
DELIMITER ;
```

Here are all three. The insert trigger logs NEW dot balance, with null for the old balance. The update trigger logs OLD dot balance and NEW dot balance. The delete trigger logs OLD dot account number and OLD dot balance, with null for new. NOW, with brackets, gives the current date and time. In the exam you write only the one they ask for.

Step 3 · Implement = TEST it

```
INSERT INTO Account VALUES (1001, 'Arun', 5000);
INSERT INTO Account VALUES (1002, 'Priya', 8000);
UPDATE Account SET Balance = 6000 WHERE AccountNo =
1001;
DELETE FROM Account WHERE AccountNo = 1002;

SELECT * FROM Account_Log;
-- LogID | AccountNo | Action | OldBalance |
NewBalance
--   1   |   1001    | INSERT |    NULL    |  5000.00
--   2   |   1002    | INSERT |    NULL    |  8000.00
--   3   |   1001    | UPDATE |  5000.00   |  6000.00
--   4   |   1002    | DELETE |  8000.00   |    NULL

SHOW TRIGGERS;
```

The question says create and implement, and implement means test it. Do an insert, an update or a delete on the main table, then select star from the log table. The log fills up automatically, and you never inserted into it yourself. That proves the trigger works. Show triggers lists all triggers. If the examiner asks for a before trigger, just change the word after to before.

Part 8 · XML Document + XSD Validation

> Part eight, the last topic: creating and validating an X M L document using an X M L schema definition, called X S D. You need three small files: the X S D, the X M L, and a short validation program.

File 1 · students.xsd (the rules)

1. element with children → complexType + sequence
2. simple element → type="xs:string"
3. maxOccurs="unbounded" → repeat

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="students">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="student"
maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name"
type="xs:string"/>
              <xs:element name="dept"
type="xs:string"/>
              <xs:element name="cgpa"
type="xs:decimal"/>
            </xs:sequence>
            <xs:attribute name="id" type="xs:integer"
use="required"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
```

```
  </xs:element>
</xs:schema>
```

The X S D file holds the rules. It starts with x s colon schema. The root element, students, has children, so it gets complex type and sequence. Inside, the student element can repeat, so max occurs equals unbounded. Student also has children, name, dept and C G P A, each with a simple type, x s string or x s decimal. And an attribute I D of type integer that is required. The pattern is: an element with children uses complex type and sequence. A simple element just gets a type.

File 2 · students.xml (the data)

1. Same element names & order as the XSD
2. noNamespaceSchemaLocation links to the XSD
3. Every tag must be closed

```
<?xml version="1.0" encoding="UTF-8"?>
<students
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:noNamespaceSchemaLocation="students.xsd">
  <student id="101">
    <name>Arun</name>
    <dept>CSE</dept>
    <cgpa>8.5</cgpa>
  </student>
  <student id="102">
    <name>Priya</name>
    <dept>IT</dept>
    <cgpa>9.1</cgpa>
  </student>
</students>
```

The X M L file holds the data. The root tag students includes no namespace schema location equals

students dot x s d, which links it to the rules file. Then each student, with the id attribute, and name, dept and C G P A tags, in the same order as the sequence in the X S D. Every opening tag needs a closing tag with a slash.

File 3 · Validate (Java — no extra library)

1. javac Validate.java
2. java Validate
3. Break the XML on purpose → shows INVALID

```
import javax.xml.XMLConstants;
import javax.xml.transform.stream.StreamSource;
import javax.xml.validation.*;
import java.io.File;

public class Validate {
    public static void main(String[] args) {
        try {
            SchemaFactory f =
SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA
_NS_URI);
            Schema schema = f.newSchema(new
File("students.xsd"));
            Validator v = schema.newValidator();
            v.validate(new StreamSource(new
File("students.xml")));
            System.out.println("XML is VALID
according to the XSD");
        } catch (Exception e) {
            System.out.println("XML is INVALID: " +
e.getMessage());
        }
    }
}
```

The validation program, in Java, with nothing extra to install. Schema factory loads the X S D, it creates a validator, and validate checks the X M L. If there's no

error, print valid. If there's an exception, print invalid with the message. Compile with javac validate dot java and run with java validate. To demonstrate properly, break the X M L on purpose, for example delete the name tag, and run it again to show the invalid message.

Option · Validate in Python (lxml)

```
from lxml import etree

schema = etree.XMLSchema(etree.parse("students.xsd"))
doc    = etree.parse("students.xml")

if schema.validate(doc):
    print("XML is VALID according to the XSD")
else:
    print("XML is INVALID:")
    for error in schema.error_log:
        print(" -", error.message)

# Restriction example in XSD (CGPA must be 0–10):
#   <xs:restriction base="xs:decimal">
#     <xs:minInclusive value="0"/> <xs:maxInclusive
value="10"/>
#   </xs:restriction>
```

If your lab uses Python instead, here's the same thing with l x m l. Parse the X S D into an X M L schema, parse the document, and call validate. It returns true or false, and the error log tells you what's wrong. At the bottom is a bonus: a restriction that forces C G P A to be between zero and ten, using min inclusive and max inclusive. That's the X S D version of a check constraint.

Part 9 · How to Attack ANY Question + Final Revision

Final part. The table names in your exam may be different from your question bank. That's fine. Here's the method to attack any question, followed by a rapid revision of every template.

The 5-step method

- 1 · Read the schema → mark PKs and FKs, find the parent and child tables
- 2 · Split the question at every full stop → each sentence = one statement
- 3 · Spot the keyword → pick the recipe (CREATE / UPDATE / GROUP BY / SAVEPOINT ...)
- 4 · Fill the template with the new table & column names
- 5 · Insert 2–3 sample rows yourself, run it, show SELECT output

The five step method. One: read the schema, and mark the primary keys and foreign keys, so you know which tables are parents and which is the child. Two: split the question at every full stop. Each sentence is usually one S Q L statement. Three: spot the keyword in each sentence and pick the recipe. Four: fill the template with the new names. Five: insert two or three sample rows yourself, because most questions need data to show output, then run it and show the select.

Example · a question you've never seen

- Library(BookID, Title, Author, Price), Member(MemberID, Name), Issue(IssueID, MemberID, BookID, IssueDate)
- Q: "Create Issue with FKs. Find books priced above average. Count issues per member.
- Issue a book, create a savepoint, issue another, roll back the second."
- → sentence 1: CREATE TABLE child + 2 FOREIGN KEY lines
- → sentence 2: WHERE Price > (SELECT AVG(Price) FROM Library)
- → sentence 3: "per" → GROUP BY MemberID with COUNT(*)
- → sentence 4: INSERT · SAVEPOINT sp · INSERT · ROLLBACK TO sp · COMMIT

> Let's try a question you've never seen: a library schema with books, members and issues. Create issue with foreign keys, find books priced above average, count issues per member, and issue a book, create a savepoint, issue another, roll back the second. Sentence one: the child table with two foreign key lines. Sentence two: greater than average, so the A V G subquery. Sentence three: per member, so group by member I D with count. Sentence four: the savepoint recipe. See? New tables, the same four recipes.

That unseen question — solved

```
CREATE TABLE Issue (
    IssueID   INT PRIMARY KEY,
```

```
    MemberID  INT,
    BookID    INT,
    IssueDate DATE,
    FOREIGN KEY (MemberID) REFERENCES
Member(MemberID),
    FOREIGN KEY (BookID)   REFERENCES Library(BookID)
);
SELECT * FROM Library WHERE Price > (SELECT
AVG(Price) FROM Library);

SELECT MemberID, COUNT(*) AS TotalIssues FROM Issue
GROUP BY MemberID;

SET autocommit = 0;
START TRANSACTION;
INSERT INTO Issue VALUES (1, 1, 10, '2025-03-01');
SAVEPOINT sp1;
INSERT INTO Issue VALUES (2, 1, 11, '2025-03-02');
ROLLBACK TO sp1;
COMMIT;
```

> And here it is, solved, purely from the templates. The create table with two foreign keys. The average subquery. Group by with count. And the savepoint sequence. If you can do this, you're ready for any variation they throw at you.

Rapid revision 1 · DDL & DML

```
CREATE TABLE T (id INT PRIMARY KEY, nm VARCHAR(50)
NOT NULL UNIQUE,
  sal DECIMAL(10,2) CHECK (sal > 20000), price
DECIMAL(6,2) DEFAULT 150,
  pid INT, FOREIGN KEY (pid) REFERENCES P(pid) ON
DELETE CASCADE);
ALTER TABLE T ADD CONSTRAINT fk1 FOREIGN KEY (pid)
REFERENCES P(pid);
INSERT INTO T VALUES (1, 'A', 30000, 200, 5);
INSERT INTO T (id, nm, sal, pid) VALUES (2, 'B',
40000, 5);  -- DEFAULT used
UPDATE T SET sal = sal * 1.10 WHERE id = 1;
DELETE FROM T WHERE id = 2;
```

Rapid revision. Pause on each of these slides if you need to. Create table with every constraint in one go, alter table add constraint for a foreign key, insert with all values, insert with a column list so the default is used, update with set and where, and delete with where.

Rapid revision 2 · SELECT & JOIN

```
SELECT col, COUNT(*) FROM T WHERE c = 'x' GROUP BY
col HAVING COUNT(*) > 1 ORDER BY col DESC;
SELECT * FROM T WHERE sal > (SELECT AVG(sal) FROM T);
SELECT * FROM P WHERE pid IN     (SELECT pid FROM T);
-- at least one
SELECT * FROM P WHERE pid NOT IN (SELECT pid FROM T);
-- never
SELECT a.x, b.y FROM A a JOIN B b ON a.id = b.id;
SELECT a.x, b.y FROM A a LEFT JOIN B b ON a.id =
b.id;
SELECT ... LEFT JOIN ...  UNION  SELECT ... RIGHT
JOIN ...;   -- full outer
SELECT * FROM A CROSS JOIN B;
SELECT e1.n, e2.n FROM E e1 JOIN E e2 ON e1.d = e2.d
AND e1.id < e2.id;
```

Revision two. The full select order with group by, having and order by. The average subquery. IN for at least one, NOT IN for never. Inner join, left join, full outer join using union, cross join, and self join. Every select question is one of these lines.

Rapid revision 3 · TCL · DCL · View · Index

```
SET autocommit = 0;
START TRANSACTION;  ...  COMMIT;
START TRANSACTION;  ...  ROLLBACK;
START TRANSACTION;  A;  SAVEPOINT sp;  B;  ROLLBACK
TO sp;  COMMIT;
```

```
CREATE USER 'u'@'localhost' IDENTIFIED BY 'pass123';
GRANT SELECT, INSERT ON db.T TO 'u'@'localhost';
REVOKE INSERT ON db.T FROM 'u'@'localhost';
GRANT ALL PRIVILEGES ON db.T TO 'u'@'localhost';

CREATE VIEW V AS SELECT ... FROM ... JOIN ... ON ...;
CREATE INDEX idx_t_col ON T(col);
```

Revision three. Autocommit zero. Commit, rollback, and the savepoint sequence. Create user, grant to, revoke from, grant all privileges. Create view as a select. Create index on table, column.

Rapid revision 4 · Procedure · Function · Trigger

```
DELIMITER //
CREATE PROCEDURE p(IN x INT)
BEGIN  SELECT * FROM T WHERE id = x;  END //

CREATE FUNCTION f(a INT, b INT) RETURNS INT
DETERMINISTIC
BEGIN  RETURN a + b;  END //

CREATE TRIGGER trg AFTER INSERT ON T FOR EACH ROW
BEGIN  INSERT INTO T_Log VALUES (NULL, NEW.id,
'INSERT', NOW());  END //
DELIMITER ;

CALL p(1);   SELECT f(2, 3);
```

Revision four. Delimiter double slash. Procedure with an IN parameter and a body. Function with returns, deterministic and return. Trigger: after insert on table, for each row, insert into the log using NEW. Delimiter semicolon. Then call the procedure and select the function.

Common errors → quick fix

- ERROR 1452 (FK fails) → insert the parent row first / value must exist
- ERROR 1062 (Duplicate entry) → primary key / unique value already used
- ERROR 3819 / 4025 (CHECK failed) → value breaks the CHECK rule
- ERROR 1064 (syntax) → missing comma, quote, bracket or ;
- ERROR 1050 (table exists) → DROP TABLE IF EXISTS t; (drop child before parent)
- ROLLBACK does nothing → you forgot SET autocommit = 0; / START TRANSACTION;

> Finally, common errors and how to fix them fast. Fourteen fifty two: foreign key fails, insert the parent first. Ten sixty two: duplicate entry, that primary key is already used. A check constraint error means your value breaks the check rule. Ten sixty four is a syntax error, look for a missing comma, quote, bracket or semicolon. Ten fifty: table already exists, drop table if exists, child before parent. And if rollback does nothing, you forgot autocommit zero or start transaction.

> That's everything. You now know every pattern in your papers. Tonight, rest and take care of your fever. Tomorrow morning, just watch the recap slides and the rapid revision part again. In the exam, read the schema, split the question into sentences, match each one to a recipe, and fill in the names. You've got this. All the best!